

Resource Allocation System – Iteration 38

Backend Application Facade / ResourceAllocationSession

Najib Attar

June 2026

1 Iteration Goal

The goal of Iteration 38 was to improve the separation between the C++ backend workflow and the user interface layer by introducing a backend application facade. The system already contained most business logic inside the C++ backend, including rules, allocation strategies, factories, participant availability checking, exam sharing, and meeting time suggestions. However, the overall execution workflow was still coordinated mainly inside `main.cpp`, with additional orchestration knowledge also appearing in the Flask dashboard.

To address this, Iteration 38 introduced a new service class called `ResourceAllocationSession`. This class provides a cleaner backend-facing workflow interface that can be reused later by `main.cpp`, Flask, a command-line interface, a REST API, a React frontend, or a desktop interface.

This iteration does not change allocation decisions. It only centralizes the application workflow.

2 Changes from Iteration 37

Iteration 37 improved meeting time suggestions by ranking valid alternatives using a least-change policy. After that, the backend had strong domain behavior, but the application workflow was still spread across multiple entry points.

In Iteration 38, the following changes were made:

- Added `ResourceAllocationSession` in the backend services layer.
- Moved the high-level workflow coordination out of `main.cpp`.
- Kept `DataController` responsible for loading and exporting data.
- Kept `AllocationService` responsible for processing requests.
- Kept rules, strategies, factories, and suggestion logic unchanged.

- Preserved the existing CSV input and output formats.
- Preserved the existing Flask dashboard behavior.

The main purpose of the change was architectural cleanup and frontend replaceability, not a new allocation feature.

3 Scenario

Before this iteration, running the backend required `main.cpp` to manually coordinate several backend components. It loaded configuration, loaded system data, created an `AllocationService`, processed requests, exported results, and cleaned up allocated memory.

This worked, but it meant that the complete backend workflow was not represented by one clear application-level interface. If another frontend replaced Flask in the future, that frontend would still need to understand several backend workflow details.

After Iteration 38, the intended scenario is:

1. A frontend or entry point creates a `ResourceAllocationSession`.
2. The session loads the selected strategy name.
3. The session loads users, spaces, requests, busy slots, and request participants.
4. The session runs allocation through the existing `AllocationService`.
5. The session exports allocations and request results.
6. The session cleans up backend data safely.

This provides one clearer backend workflow boundary.

4 Modules and Their Tasks

Module	Task
<code>ResourceAllocationSession</code>	Coordinates the high-level backend workflow. It loads strategy configuration, loads system data, runs allocation, exports results, exposes allocations and system data, and performs cleanup.
<code>DataController</code>	Continues to load users, spaces, requests, user busy slots, and request participants. It also exports allocation and request-result CSV files.
<code>AllocationService</code>	Continues to process requests using the selected allocation strategy and rule engine facade. It also appends committee meeting suggestions when needed.
<code>AllocationStrategyFactory</code>	Continues to select the requested strategy, such as greedy, priority, multi-room exam, or shared-room exam best-fit.
<code>RuleEngineFacade</code>	Continues to coordinate request validation rules and physical room availability checks.
<code>main.cpp</code>	Becomes thinner. It now delegates workflow steps to <code>ResourceAllocationSession</code> and mainly remains responsible for console printing.

Table 1: Main modules affected by Iteration 38

5 ResourceAllocationSession Design

The new `ResourceAllocationSession` class acts as an application facade. It does not replace existing backend components. Instead, it coordinates them.

The session stores:

- a `DataController`
- loaded `SystemData`
- the selected strategy name
- an `AllocationService`
- status flags for whether data is loaded and allocation has run

The main public methods include:

- `loadStrategyName(...)`

- `loadSystemData(...)`
- `runAllocation()`
- `exportResults(...)`
- `getAllocations()`
- `getSystemData()`
- `cleanup()`

This design keeps the class focused on coordination. It does not contain rule evaluation logic, allocation algorithms, request creation logic, user creation logic, or meeting suggestion logic.

6 Iteration Comparison

Area	Before Iteration 38	After Iteration 38
Backend workflow	Mostly coordinated directly inside <code>main.cpp</code> .	Coordinated through <code>ResourceAllocationSession</code> .
Frontend replacement	Possible, but another frontend would need to understand several workflow details.	Easier, because a single backend session class exposes the main workflow.
Allocation logic	Implemented in strategies and <code>AllocationService</code> .	Unchanged.
Rule logic	Implemented in rule classes and <code>RuleEngineFacade</code> .	Unchanged.
Data loading/exporting	Implemented in <code>DataController</code> .	Unchanged, but now called through the session workflow.
Flask dashboard	Runs backend executable and reads CSV outputs.	Unchanged in this iteration.
CSV formats	Existing input and output formats.	Unchanged.

Table 2: Comparison between Iteration 37 and Iteration 38

7 Updated Class Diagram

The updated class diagram should show the new facade layer. The most important new relationship is that `main.cpp` uses `ResourceAllocationSession`, and the session coordinates existing backend components.

The diagram should include:

- `ResourceAllocationSession`
- `DataController`
- `AllocationService`
- `SystemData`
- `Allocation`

It should show that `ResourceAllocationSession` uses `DataController` for loading/exporting, owns or manages loaded `SystemData`, uses `AllocationService` to process requests, and exposes generated `Allocation` objects.

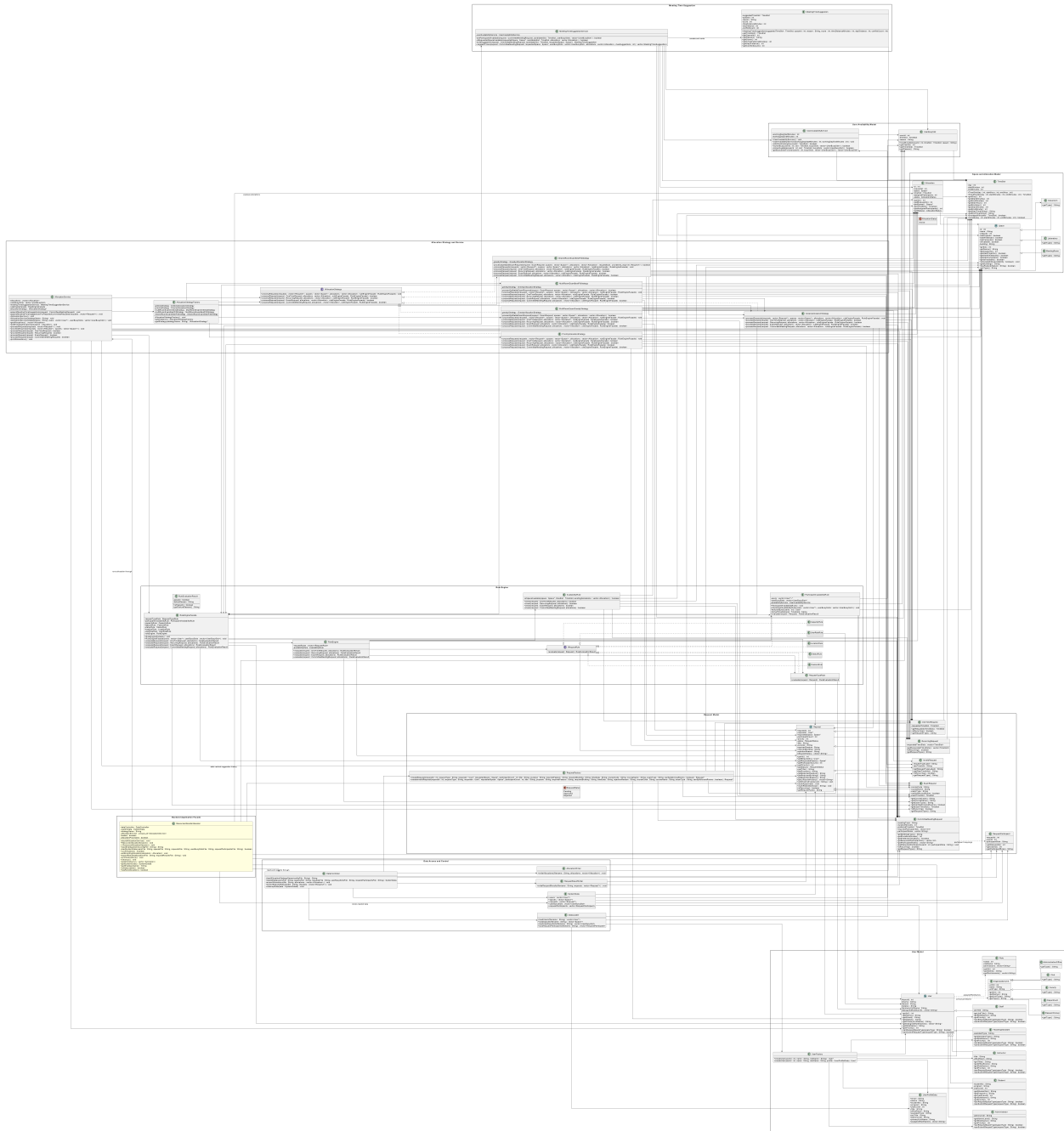


Figure 1: Updated class diagram for Iteration 38

8 Updated Workflow Diagram

The updated workflow diagram should focus on the new application facade workflow rather than redrawing all internal rule and strategy details.

The workflow should show:

1. `main.cpp` creates a `ResourceAllocationSession`.

2. The session loads the strategy from `data/config.txt`.
3. The session loads users, spaces, requests, busy slots, and participants.
4. The session creates or rebuilds `AllocationService`.
5. The session runs allocation.
6. Existing rules and strategies process the requests.
7. The session exports `allocations.csv` and `request_results.csv`.
8. The session performs cleanup.

9 Prototype Implementation

The implementation added two new backend files:

- `src/services/ResourceAllocationSession.h`
- `src/services/ResourceAllocationSession.cpp`

The implementation also updated `src/main.cpp`. The main function now creates a session, loads configuration, loads data, runs allocation, exports results, and cleans up through the session object. Console output for individual request summaries remains in `main.cpp`, because that output is presentation-related rather than core backend workflow logic.

A regression test file was also added:

- `tests/test_resource_allocation_session.cpp`

The test verifies that a `ResourceAllocationSession` can load temporary CSV data, run allocation, export output files, and clean up safely. It also verifies that failed data loading is handled without running allocation.

The backend test command used was:

```
powershell -ExecutionPolicy Bypass -File tests\run_tests.ps1
```

The test suite passed with:

58 passed, 0 failed

10 What Did Not Change

This iteration intentionally avoided changing domain behavior. The following parts remain unchanged:

- allocation strategies
- rule evaluation behavior
- request factory behavior
- user factory behavior
- committee meeting suggestion algorithm
- least-change ranking logic
- exam sharing and multi-room exam behavior
- CSV input and output formats
- Flask dashboard workflow

Therefore, Iteration 38 is an architectural coordination improvement, not a business-rule or algorithm iteration.

11 Remaining Limitation and Future Work

Iteration 38 improves backend workflow centralization, but it does not fully remove all UI coupling. The Flask dashboard may still know CSV column names, parse backend output files, and display suggestion text from exported request history.

Future work may include:

- adding a structured backend summary export
- reducing Flask dependence on raw CSV schemas
- exposing the session workflow through a REST API
- allowing a future React or desktop frontend to call one stable backend interface
- separating UI formatting from backend result representation more clearly

These improvements are intentionally left for later iterations.

12 Summary

Iteration 38 introduced `ResourceAllocationSession` as a backend application facade. This class centralizes the main backend workflow while preserving the existing architecture. It coordinates data loading, strategy selection, allocation execution, result exporting, and cleanup using existing backend components.

The iteration makes `main.cpp` thinner and prepares the system for future frontend replacement. It does not change allocation algorithms, validation rules, factories, CSV formats, or Flask behavior. The main benefit is a clearer boundary between application workflow coordination and the rest of the backend domain logic.